

GITHUB COPILOT

**Agente autónomo
en la nube**

GitHub Copilot Cloud Agent

Habilitación, seguridad, asignación de tareas, personalización
con MCP y uso responsable

Agenda

01 Entender y habilitar

Qué es, disponibilidad, tareas, formas de delegar y costos en Actions y PRUs.

02 Seguridad y límites

Modelo de gobernanza, riesgos, mitigaciones y limitaciones conocidas.

03 Asignar y hacer seguimiento

Issues, API GraphQL, señales de progreso, iteración y troubleshooting.

04 Personalizar y validar

Entorno previo, runners, firewall, MCP y prácticas de testing.

05 Uso responsable

Funcionamiento extremo a extremo, casos de uso y limitaciones generales.

Cierre

Puntos clave para adoptar el agente con control y calidad.

01

Entender y habilitar el Cloud Agent

Qué hace el agente, dónde está disponible y qué recursos consume

DEFINICIÓN

Un integrante autónomo del equipo dentro de GitHub

- Asistente autónomo de desarrollo que recibe tareas claramente definidas.
- Explora el código, crea una rama y genera un plan de implementación.
- Prepara los cambios manteniendo al usuario en control de cuándo abrir un Pull Request.
- Al finalizar el trabajo, solicita revisión al equipo.
- Mencionar @copilot en un comentario del Pull Request le permite iterar sobre su trabajo.

Trabajo visible

Logs de sesión e historial de Pull Requests.

Control humano

El usuario decide si y cuándo abrir el PR.

Colaboración

Los compañeros ven los pasos y participan.

Disponibilidad

Copilot Pro

Plan individual.

Copilot Pro+

Plan individual ampliado.

Copilot Business

Plan de organización.

Copilot Enterprise

Plan de empresa.

Repositorios

- Repositorios alojados en GitHub.
- Excepción: repositorios pertenecientes a cuentas de usuario administradas.
- Excepción: repositorios donde el agente está deshabilitado.

Tareas que puede realizar

Corregir bugs

Errores y regresiones detectadas en el código.

Nuevas funcionalidades

Implementaciones incrementales bien delimitadas.

Cobertura de tests

Mejorar la cobertura o generar tests faltantes.

Documentación

Crear o actualizar documentación del proyecto.

Deuda técnica

Resolver pendientes y refactorizaciones específicas.

Prototipado

Crear nuevos proyectos y conceptos iniciales.

Formas de delegar trabajo

Asignar un issue a Copilot

Desde GitHub.com, GitHub Mobile o mediante API y CLI. El agente recibe título, descripción y comentarios existentes.

Solicitar cambios de código

Desde el panel Agents, Copilot Chat, el IDE, otra herramienta agéntica compatible con MCP o Raycast en macOS.

Al finalizar, el agente solicita revisión. Un comentario en el Pull Request mencionando @copilot le pide iterar sobre el trabajo entregado.

Diferencias con los asistentes tradicionales del IDE

Asistente tradicional

- Ayuda a escribir código localmente.
- El desarrollador crea ramas y hace commits.
- El push y la descripción del PR son manuales.
- La iteración ocurre fuera del historial del repositorio.

Copilot Cloud Agent

- Todo el trabajo ocurre mediante commits en GitHub.
- Automatiza ramas, mensajes de commit y generación de código.
- El usuario decide si y cuándo abrir el Pull Request.
- El trabajo queda visible en logs de sesión e historial de PRs.
- La interacción se realiza con comentarios de revisión del PR.

Cloud Agent frente a Agent Mode del IDE

Cloud Agent

Funciona autónomamente en un entorno basado en GitHub Actions para completar tareas asignadas mediante issues o Copilot Chat.

Agent Mode (Copilot Edits)

Realiza ediciones autónomas de forma local, dentro de la sesión del IDE del desarrollador.

Ambos modos son complementarios: el trabajo local acelera la edición inmediata y el agente en la nube absorbe tareas asíncronas y delimitadas.

Habilitación

Repositorios de una organización

La disponibilidad es administrada por los administradores de la organización o de la empresa.

Repositorios personales

El agente se configura desde las opciones de la cuenta del usuario.

Las políticas de seguridad y gobernanza existentes continúan aplicándose una vez habilitado el agente.

Costos: GitHub Actions y PRUs

GitHub Actions minutes

Sostienen el entorno temporal de compilación y pruebas donde trabaja el agente.

Copilot Premium Request Units

Cubren el razonamiento de los modelos avanzados que utiliza el agente.

- Desde el 4 de junio de 2025 el agente utiliza una solicitud premium por cada solicitud de modelo que realiza.
- Dentro de las asignaciones mensuales de Actions y solicitudes premium, las tareas pueden ejecutarse sin cargos adicionales.
- Conviene reservar las PRUs para ediciones de múltiples archivos, generación de tests y cambios amplios que exigen mayor razonamiento.
- Las modificaciones simples requieren menos pasos intensivos en PRUs.

02

Seguridad, riesgos y limitaciones

Gobernanza, entorno restringido, mitigaciones y límites conocidos

Modelo de seguridad y protecciones

Gobernanza

La disponibilidad se controla desde la organización y la empresa; las políticas existentes siguen vigentes.

Entorno restringido

Sandbox de GitHub Actions con acceso a Internet protegido por firewall y lectura del repositorio.

Restricción de ramas

Solo crea y hace push a ramas copilot/; las protecciones y comprobaciones requeridas se mantienen.

Permisos

Solo responde a usuarios con permisos de escritura; los demás comentarios se ignoran.

Colaboradores externos

Los PRs del agente requieren aprobación de un usuario con escritura antes de ejecutar Actions.

Cumplimiento y atribución

Los commits se realizan junto al desarrollador que asignó la tarea, manteniendo la atribución clara.

Riesgo: el agente puede hacer push de código

- 1 Solo los usuarios con permisos de escritura pueden activar trabajos del agente.
- 2 Los pushes están restringidos a ramas copilot/, nunca a main o master.
- 3 Las credenciales del agente permiten únicamente un push simple.
- 4 Los workflows de Actions no se ejecutan hasta seleccionar Approve and run workflows.
- 5 El solicitante del Pull Request no puede aprobarlo por sí mismo.

La revisión humana del resultado sigue siendo obligatoria, igual que con el código de cualquier integrante del equipo.

Información sensible y prompt injection

Acceso a información sensible

El acceso a Internet del agente está restringido mediante firewall de forma predeterminada. Puede personalizarse o deshabilitarse según la política de la organización.

Prompt injection

Los caracteres ocultos, como los comentarios HTML, se filtran antes de pasar la entrada al agente, reduciendo el riesgo de instrucciones maliciosas ocultas en comentarios o issues.

A pesar de estas protecciones, los resultados deben revisarse cuidadosamente antes de integrarlos.

Limitaciones del workflow

- Solo puede realizar cambios en el mismo repositorio donde se encuentra el issue o Pull Request asignado.
- Por defecto el contexto está limitado al repositorio asignado; puede ampliarse mediante MCP.
- Abre exactamente un Pull Request por tarea.
- No puede modificar un Pull Request existente que no haya creado.

Limitaciones de compatibilidad

- No firma commits: si se requieren commits firmados, hay que reescribir el historial antes del merge.
- Requiere runners Ubuntu x64 alojados en GitHub y no admite self-hosted runners.
- No está disponible para repositorios personales de cuentas de usuario administradas.
- No respeta las exclusiones de contenido: puede ver y actualizar archivos excluidos.
- La política Suggestions matching public code no se aplica y pueden no proporcionarse referencias.
- Funciona únicamente con repositorios alojados en GitHub.
- No se puede cambiar el modelo de IA utilizado; lo selecciona GitHub.

03

Asignar, seguir y solucionar tareas

Del issue al Pull Request revisado, con señales claras de progreso

Qué ocurre al asignar un issue a Copilot

- 1 Copilot agrega una reacción de observación al issue.
- 2 Crea una rama dedicada con el prefijo copilot/.
- 3 Abre un Pull Request en estado draft vinculado al issue.
- 4 Inicia una sesión del agente en un entorno basado en GitHub Actions.
- 5 Realiza commits en la rama y actualiza el cuerpo del PR con mensajes de estado.
- 6 Publica el evento Copilot finished work y solicita revisión.

Dónde y cómo asignar

Desde la interfaz

- Issues, luego Assignees y Copilot en GitHub.com.
- La lista de issues del repositorio.
- GitHub Projects.
- GitHub Mobile.
- GitHub CLI mediante `gh issue edit`.

Copilot recibe el título, la descripción y los comentarios existentes al momento de la asignación; la información nueva debe agregarse como comentario en el PR del agente.

Asignación mediante GraphQL API

- Verificar la disponibilidad consultando `suggestedActors` y comprobando `copilot-swe-agent`.
- Obtener el ID del repositorio.
- Crear y asignar con `createIssue` usando el ID del repositorio y el del bot.
- Asignar un issue existente con `replaceActorsForAssignable`.

Seguimiento del progreso

Confirmación inmediata

El agente reacciona al issue para indicar que recibió la tarea.

Creación del PR

Abre un draft Pull Request vinculado al issue original.

Sesión activa

Aparece Copilot started work en la timeline; actualiza el cuerpo del PR y hace commits.

Logs de sesión

Desde la página Agents, View session muestra las acciones en tiempo real y Stop session detiene la ejecución.

Finalización

Aparece Copilot finished work y se solicita una revisión.

Historial

Sesiones pasadas y actuales quedan disponibles para consulta.

Iterar con Copilot

1 Mencionar @copilot en un comentario del Pull Request para solicitar cambios.

2 Solo se procesan comentarios de usuarios con permisos de escritura.

3 Copilot reacciona al comentario para confirmar que recibió la solicitud.

4 Aparece Copilot started work en la timeline mientras retoma la tarea.

Aprobaciones y workflows

- Los Pull Requests creados por Copilot siempre están en estado draft.
- Requieren aprobación humana antes del merge.
- Los workflows de GitHub Actions no se ejecutan automáticamente.
- Para ejecutarlos se debe seleccionar Approve and run workflows en el cuadro de merge.
- El desarrollador que solicitó la creación del PR no puede aprobarlo.

Así se mantienen las reglas de revisiones requeridas y se garantiza una revisión independiente del trabajo del agente.

Troubleshooting: acceso y asignación

Copilot no aparece en Assignees

Verificar que el plan sea Pro, Pro+, Business o Enterprise y que el agente no esté deshabilitado en la organización o el repositorio.

Repositorios personales EMU

El agente no está disponible; utilizar repositorios propiedad de una organización.

Cannot create a pull request desde Chat

Comprobar la disponibilidad del agente y, en IDEs, mencionar @github en el prompt.

Se asignó un issue y no ocurre nada

Actualizar la página y buscar la reacción del agente y, después, el draft Pull Request.

Troubleshooting: ejecución y entorno

PR creado pero sin progreso

Revisar la timeline buscando Copilot started work y abrir los logs de sesión.

No responde a un comentario

Comprobar permisos de escritura y que se haya mencionado @copilot en el PR del agente.

El agente parece bloqueado

Las sesiones se recuperan y tienen timeout de una hora; reasignar el issue o volver a publicar el comentario.

Actions no se ejecutan

Seleccionar Approve and run workflows en el cuadro de merge.

Los pushes no pasan CI

Dar instrucciones claras en [.github/copilot-instructions.md](#) para validar con tests y linters.

Firewall e imágenes

Las advertencias muestran dirección y comando bloqueados; las imágenes superiores a 3,00 MiB se eliminan.

04

Personalizar, extender y validar

Entorno previo, MCP y prácticas de testing antes del merge

Preconfigurar el entorno de desarrollo

- Crear .github/workflows/copilot-setup-steps.yml en la rama predeterminada del repositorio.
- Definir un único job llamado copilot-setup-steps con los pasos para instalar dependencias y configurar herramientas.
- El workflow puede ejecutarse de forma independiente para validarlo.
- Después se ejecuta automáticamente antes de iniciar el agente, mejorando confiabilidad y velocidad.

```
jobs:
  copilot-setup-steps:
    runs-on: ubuntu-latest
    permissions:
      contents: read
    steps:
      - name: Checkout code
        uses: actions/checkout@v5
      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: "20"
      - run: npm ci
```

Claves permitidas, runners y Git LFS

Claves de configuración admitidas

- steps
- permissions
- runs-on
- container
- services
- snapshot
- timeout-minutes, hasta 59

El fetch-depth de actions/checkout se sobrescribe para permitir un rollback seguro.

Larger GitHub-hosted runners

Agregar primero los runners de mayor tamaño y configurar runs-on con el label o grupo, por ejemplo ubuntu-4-core. Solo Ubuntu x64; sin self-hosted runners.

Git LFS

Si el repositorio usa Large File Storage, habilitarlo en los pasos con actions/checkout y lfs activado.

Firewall

El acceso a Internet está limitado por defecto para reducir el riesgo de exfiltración; puede personalizarse o deshabilitarse.

Extender el agente con MCP

- MCP es un estándar abierto para conectar modelos de lenguaje con herramientas y datos.
- El agente puede utilizar herramientas de servidores MCP locales o remotos para ampliar sus capacidades.
- Admite herramientas MCP, pero no recursos ni prompts.
- Los servidores MCP remotos que requieren OAuth no son compatibles.

GitHub MCP Server

Accede a issues, Pull Requests y datos de GitHub mediante un token de solo lectura limitado al repositorio actual, personalizable.

Playwright MCP Server

Permite leer, interactuar y tomar capturas de páginas web accesibles desde el entorno del agente, incluido localhost.

Configuración y buenas prácticas de MCP

- Los administradores declaran los servidores MCP mediante una configuración JSON en el repositorio.
- Una vez configurados, el agente usa las herramientas disponibles de forma autónoma, sin aprobación en cada uso.

Revisar terceros

Evaluar rendimiento y calidad de los resultados de cada servidor.

Preferir lectura

Priorizar herramientas de solo lectura sobre las de escritura.

Escritura mínima

Permitir únicamente las herramientas de escritura necesarias.

Validar antes de guardar

Revisar cuidadosamente la configuración MCP antes de aplicarla.

Testing y validación del resultado

- Ejecutar CI con tests, linters y análisis de seguridad en cada Pull Request generado por el agente.
- Estos checks no se ejecutan hasta seleccionar Approve and run workflows.
- Inspeccionar manualmente las áreas de alto impacto o sensibles.
- Solicitar al agente la generación de tests siguiendo el estilo del repositorio.
- Utilizar rulesets para exigir que los PRs superen tests, scanning y linting antes del merge.
- Etiquetar los PRs del agente para facilitar monitoreo, clasificación y reversión.
- Ajustar .github/copilot-instructions.md cuando se detecten errores repetitivos.
- Revertir rápidamente los cambios cuando sea necesario y pedir nuevas modificaciones.

DÓNDE APORTAN MÁS VALOR

Uso intencional de las PRUs

Cobertura de tests

Ampliar suites de pruebas en múltiples archivos.

Auditorías amplias

Revisiones que abarcan varios directorios del repositorio.

Áreas de riesgo

Análisis profundo de componentes sensibles o críticos.

Las verificaciones ligeras consumen menos PRUs, por lo que conviene reservarlas de forma intencional para las validaciones que realmente exigen mayor razonamiento.

05

Uso responsable en GitHub.com

Cómo funciona de extremo a extremo y cómo obtener mejores resultados

Funcionamiento de extremo a extremo

- 1 Procesamiento del prompt: la tarea del issue, comentario o Chat se combina con contexto relevante.
- 2 Las entradas pueden incluir lenguaje natural, fragmentos de código e imágenes.
- 3 Análisis mediante el modelo de lenguaje, que razona sobre la tarea y usa las herramientas necesarias.
- 4 Generación de respuesta con sugerencias en lenguaje natural y de código.
- 5 Formateo de resultados: actualiza la descripción del PR con los cambios y las limitaciones encontradas.
- 6 El feedback del usuario vuelve al modelo y el agente responde con los cambios actualizados.

GitHub realiza red teaming como parte de su proceso de desarrollo para comprender y mejorar la seguridad del agente.

Casos de uso

Mantenimiento del código

Correcciones de seguridad, actualización de dependencias y refactorizaciones específicas.

Documentación

Actualización y creación de documentación del proyecto.

Desarrollo de funcionalidades

Implementación de solicitudes incrementales bien delimitadas.

Cobertura de tests

Desarrollo de suites adicionales de pruebas.

Prototipado

Creación de nuevos proyectos y conceptos.

Entorno efímero

Modifica código, ejecuta tests automatizados y linters durante el trabajo.

Mejorar el rendimiento y personalizar el contexto

Tareas bien delimitadas

- Una descripción clara del problema o del trabajo requerido.
- Criterios de aceptación completos sobre cómo debe ser una solución correcta.
- Indicaciones sobre los archivos que deben modificarse.

Personalizar el contexto

- Custom Copilot instructions para explicar cómo construir, probar y validar los cambios.
- El entorno de desarrollo previo del agente.
- La configuración del firewall.
- Las capacidades disponibles mediante MCP.

El inglés es el idioma principal soportado tras la evaluación del agente en diversos lenguajes de programación.

Medidas de seguridad y desarrollo seguro

Evitar escalación de privilegios

Solo responde a usuarios con permisos de escritura, los workflows requieren aprobación y los caracteres ocultos se filtran.

Limitar permisos

Accede solo a su repositorio, hace push únicamente a ramas copilot/ y no accede a los secrets de la organización.

Prevenir la exfiltración

El firewall está habilitado por defecto para evitar la salida accidental o maliciosa de código e información.

Prácticas de desarrollo seguro al revisar el código generado

- Evitar secretos hardcodeados y prevenir vulnerabilidades de inyección.
- Realizar testing riguroso y ejecutar análisis de propiedad intelectual y de vulnerabilidades.
- Revisar y probar siempre el contenido generado antes del merge.

Limitaciones generales y feedback

- Alcance y calidad limitados: el modelo puede tener dificultades con estructuras poco comunes y la calidad varía según la cobertura del lenguaje.
- Sesgos potenciales: los datos de entrenamiento y el contexto recuperado pueden inclinar al agente hacia determinados lenguajes o estilos.
- Riesgos de seguridad: el código generado podría exponer información sensible si no se revisa de forma exhaustiva.
- Código inexacto: puede parecer correcto pero ser semántica o sintácticamente incorrecto o no coincidir con la intención.
- Código público: puede producir coincidencias con código público incluso con Block configurado y sin proporcionar referencias.
- Aspectos legales y regulatorios: se debe garantizar el cumplimiento y evitar usos prohibidos por los términos de servicio.

El feedback se comparte con el botón de thumbs-down debajo de una respuesta del agente o en el foro de discusión de la comunidad, y conviene monitorear nuevos riesgos y buenas prácticas a medida que el agente evoluciona.

Puntos clave

Autonomía con control

El agente trabaja solo, pero el humano decide el PR y el merge.

Tareas bien delimitadas

Descripción clara, criterios de aceptación y archivos objetivo.

Validación siempre

CI, revisión manual y rulesets antes de integrar los cambios.

Seguridad por diseño

Sandbox, ramas copilot/, firewall y permisos de escritura.

Entorno preparado

copilot-setup-steps.yml y MCP mejoran confiabilidad y alcance.

Consumo intencional

Reservar las PRUs para el trabajo que exige más razonamiento.